Searching-Sorting-Hashing

Searching Algorithm

- Search algorithms are designed to check or retrieve an element from any data structure where that element is being stored.
- They search for a target (key) in the search space.

Types of Search Algorithms

- Linear or Sequential Search
- Binary Search

Linear or Sequential Search

- This algorithm works by sequentially iterating through the whole array or list from one end until the target element is found.
- If the element is found, it returns its index, else -1

Example:

1st iter -> idx:0 -> arr = [2, 12, 15, 11, 7]

2nd iter -> idx:1 -> arr = [2, 12, 15, 11, 7]

3rd iter -> idx:2 -> arr = [2, 12, 15, 11, 7]

4th iter -> idx:3 -> arr = [2, 12, 15, 11, 7] break

Approach for Linear or Sequential Search

Suppose the target element we want to search is 11.

- Start with index 0 and compare each element with the target
- If the target is found to be equal to the element, return its index
- If the target is not found, return -1

Binary Search

- This type of searching algorithm is used to find the position of a specific value contained **in a sorted array**.
- The binary search algorithm works on the principle of divide and conquer and it is considered the best searching algorithm because it's faster to run.

Example

- arr = [2, 12, 15, 17, 27, 29, 45]
- target is 17

Steps:

1st iter -> [**2**, 12, **15**, 17, **27**]

left ele idx = 0

right ele = 4

mid = (0+4)/2 = 2

Where mid element 15 < target element 17 so ignoring all elements in the left side,

remaining elements [~~2,12,15~~,17,27]

Steps

2nd iter -> [~~2,12,15,~~17, 27]

left ele idx = 3

right ele = 4

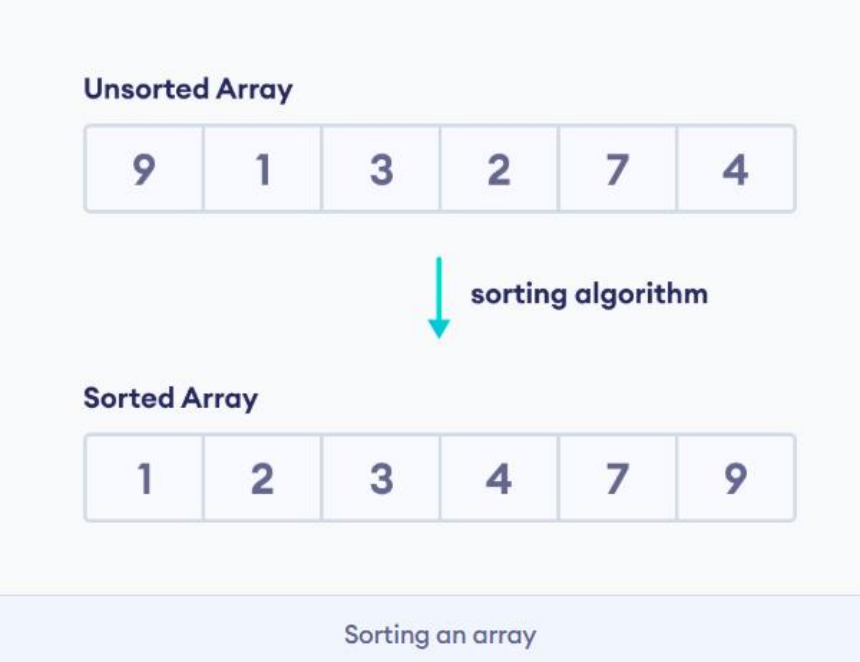
mid = (3+4)/2 = 3

mid element 17 = target element 17,return 3

Sorting Algorithm

A sorting algorithm is used to arrange elements of an array/list in a specific order.

For example, Here, we are sorting the array in ascending order.



Different Sorting Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Radix Sort
- Shell Sort

Bubble Sort

- Bubble sort is a sorting algorithm that compares two adjacent elements and swaps them until they are in the intended order.
- Just like the movement of air bubbles in the water that rise up to the surface, each element of the array move to the end in each iteration. Therefore, it is called a bubble sort.

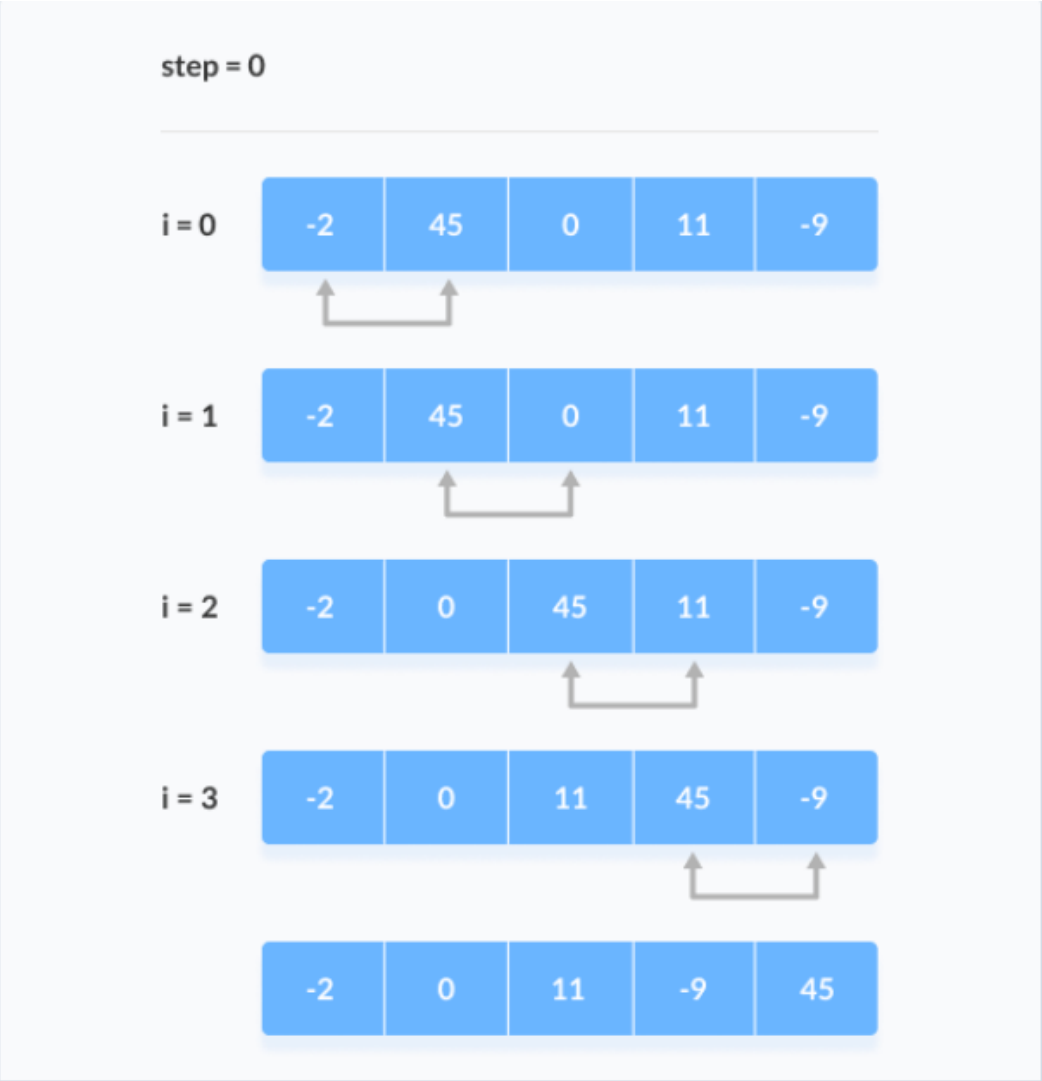
Working of Bubble Sort

Suppose we are trying to sort the elements in **ascending order**.

1. First Iteration (Compare and Swap)

- Starting from the first index, compare the first and the second elements.
- If the first element is greater than the second element, they are swapped.
- Now, compare the second and the third elements. Swap them if they are not in order.
- The above process goes on until the last element

Working of Bubble Sort



Compare the Adjacent Elements

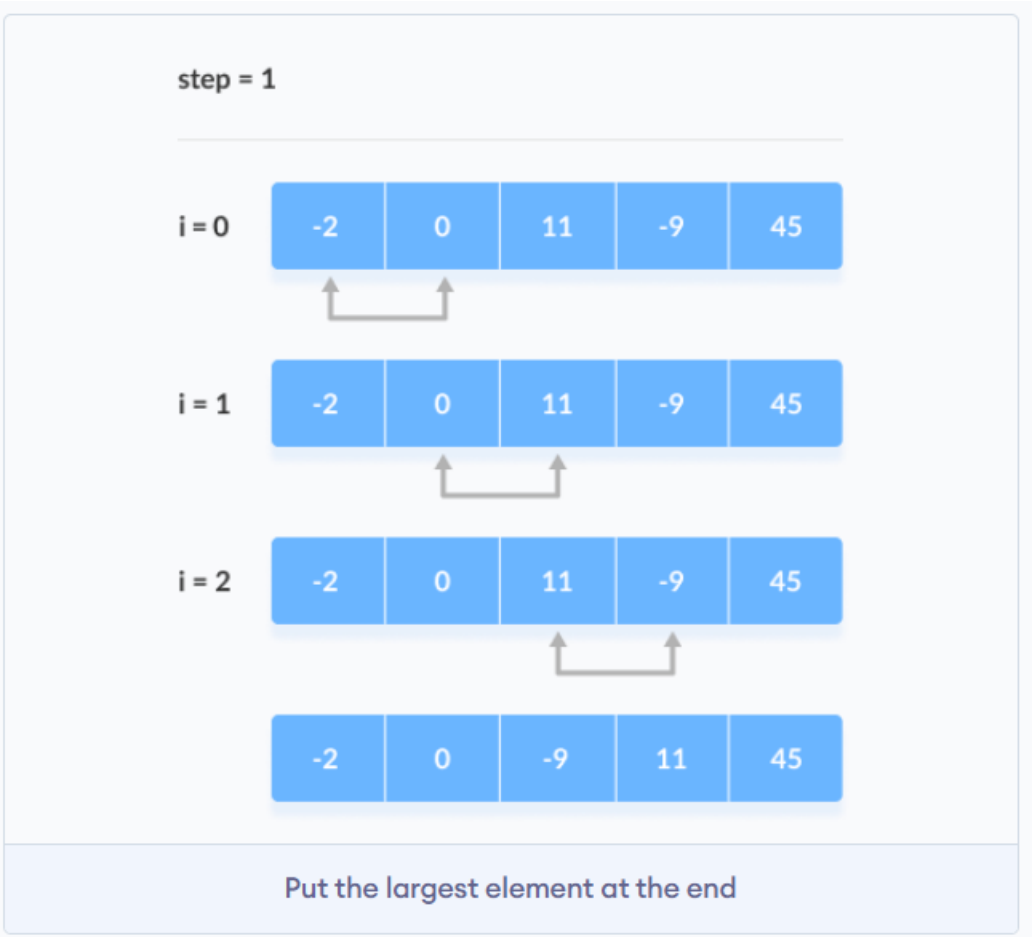
Working of Bubble Sort

2. Remaining Iteration

The same process goes on for the remaining iterations.

- After each iteration, the largest element among the unsorted elements is placed at the end.

Working of Bubble Sort



Working of Bubble Sort

- In each iteration, the comparison takes place up to the last unsorted element.
- The array is sorted when all the unsorted elements are placed at their correct positions.

Algorithm

```
bubbleSort(array)
  for i <- 1 to sizeOfArray - 1
    for j <- 1 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
end bubbleSort
```

Selection Sort Algorithm

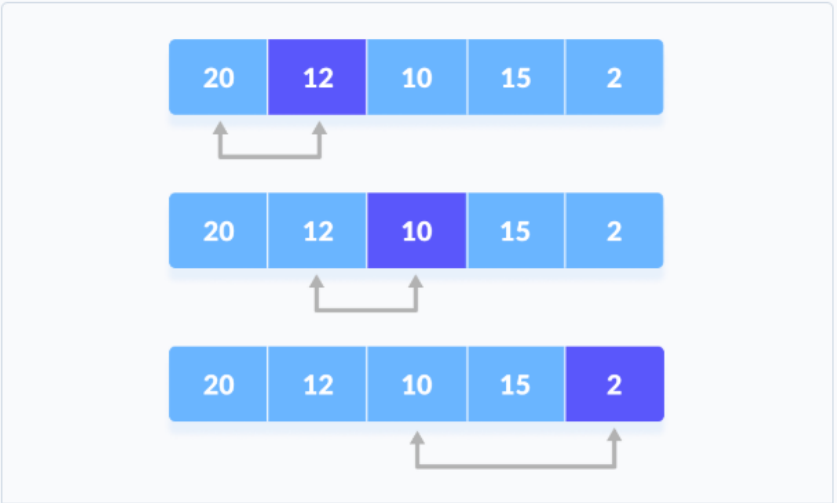
- Selection sort is a sorting algorithm that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted list.

Working of selection sort

- Set the first element as minimum.

Working of selection sort

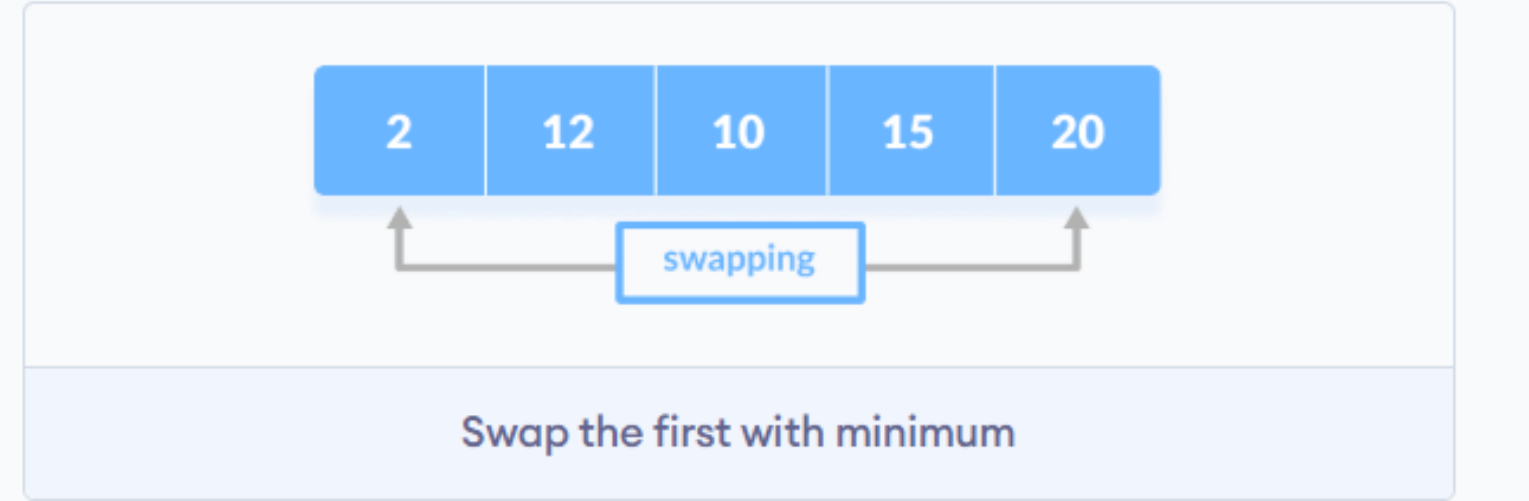
- Compare minimum with the second element. If the second element is smaller than minimum, assign the second element as minimum.
- Compare minimum with the third element. Again, if the third element is smaller, then assign minimum to the third element otherwise do nothing. The process goes on until the last element.



Compare minimum with the remaining elements

Working of selection sort

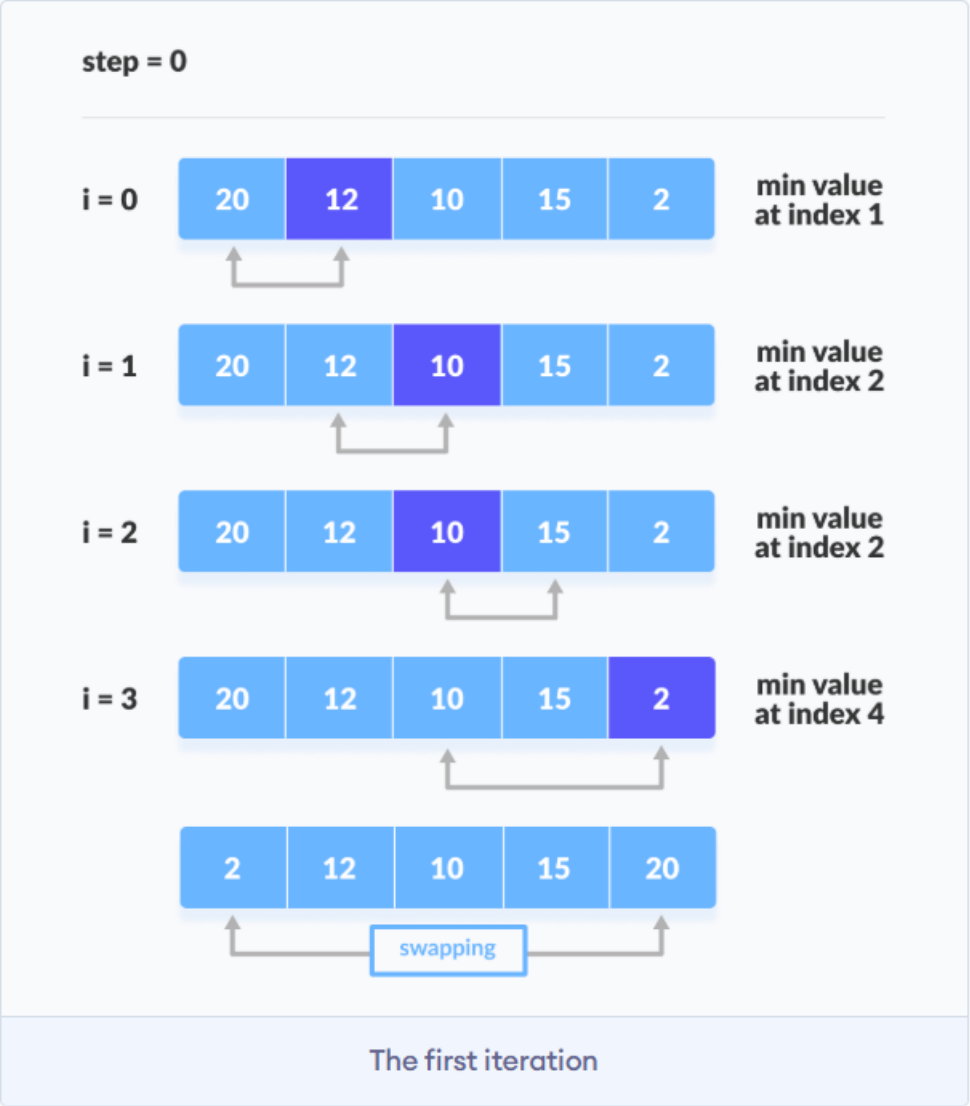
- After each iteration, minimum is placed in the front of the unsorted list.



Working of selection sort

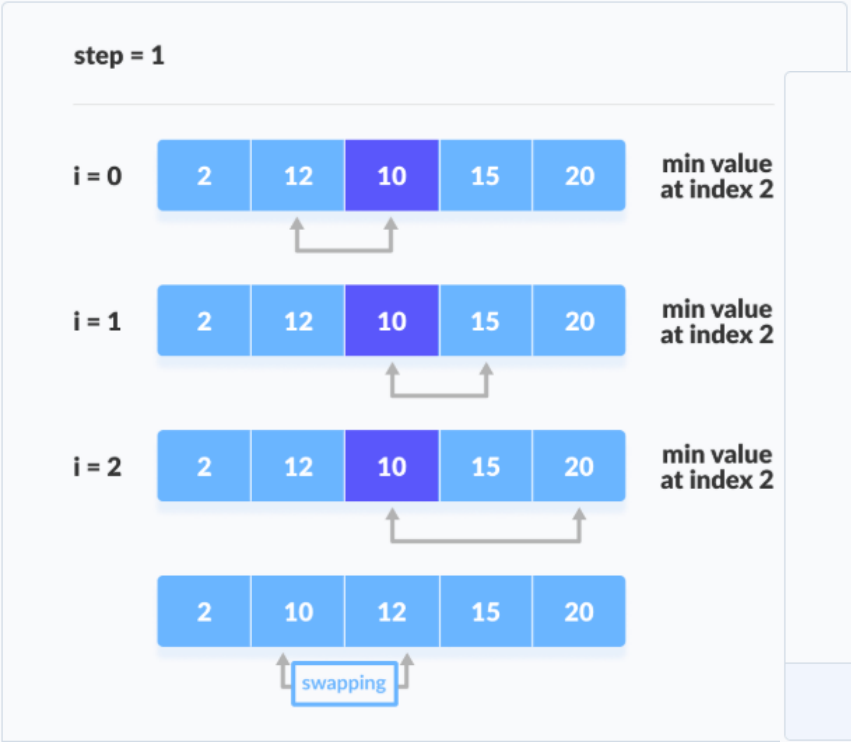
- For each iteration, indexing starts from the first unsorted element.

Step 1 to 3 are repeated until all the elements are placed at their correct positions

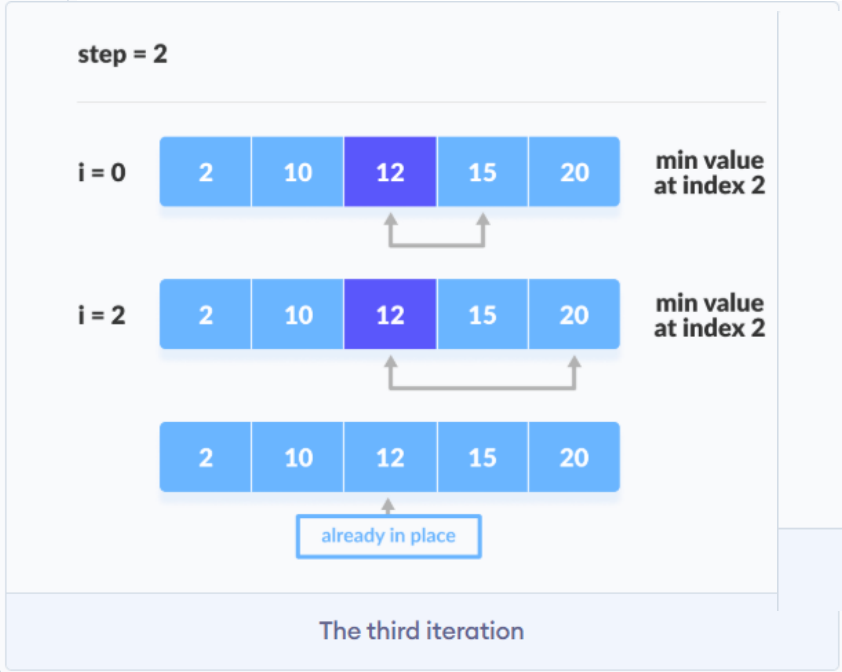


The first iteration

Working of selection sort



The second iteration



The third iteration

The fourth iteration

Selection Sort Algorithm

```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size - 1 do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
end selectionSort
```

Insertion Sort Algorithm

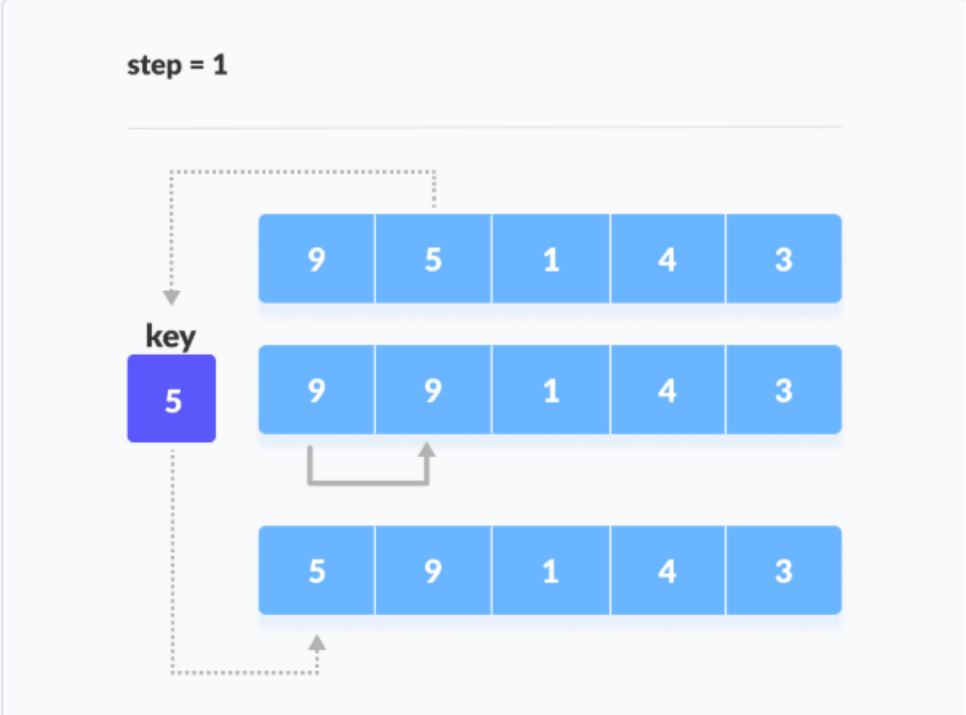
- Insertion sort is a sorting algorithm that places an unsorted element at its suitable place in each iteration.
- Insertion sort works similarly as we sort cards in our hand in a card game.

Working of Insertion Sort

Suppose we need to sort the following array.

Working of Insertion Sort

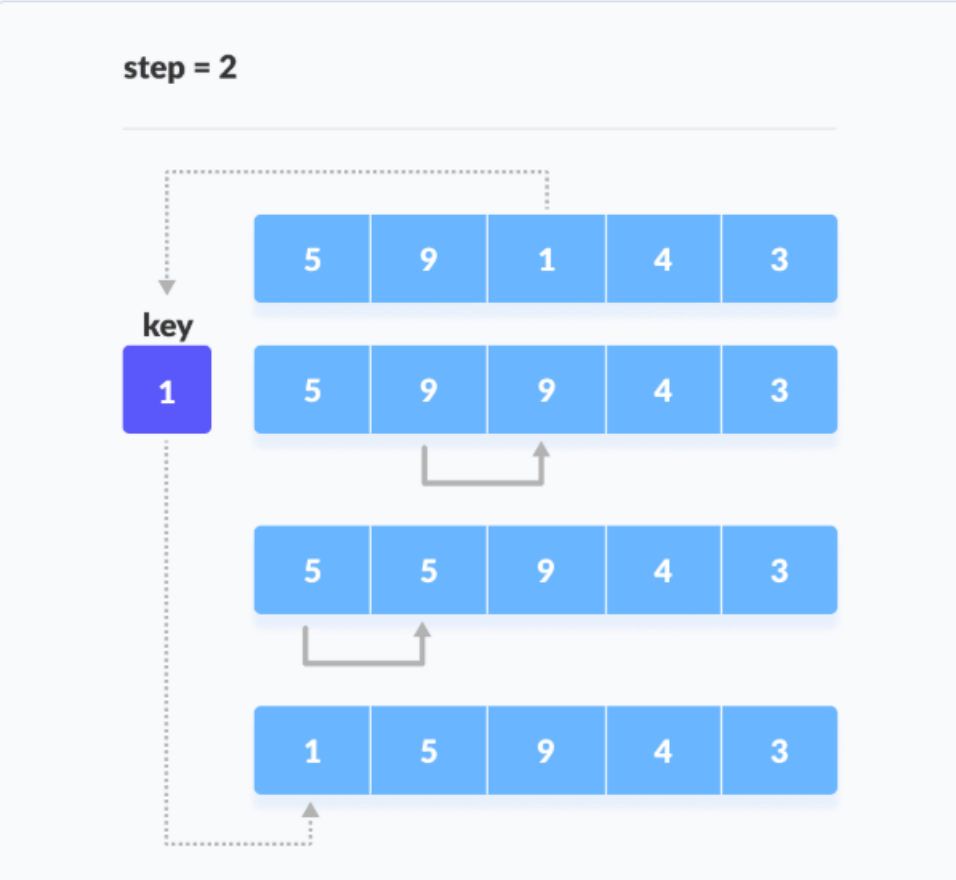
- The first element in the array is assumed to be sorted. Take the second element and store it separately in key.
- Compare key with the first element. If the first element is greater than key, then key is placed in front of the first element.



If the first element is greater than key, then key is placed in front of the first element.

Working of Insertion Sort

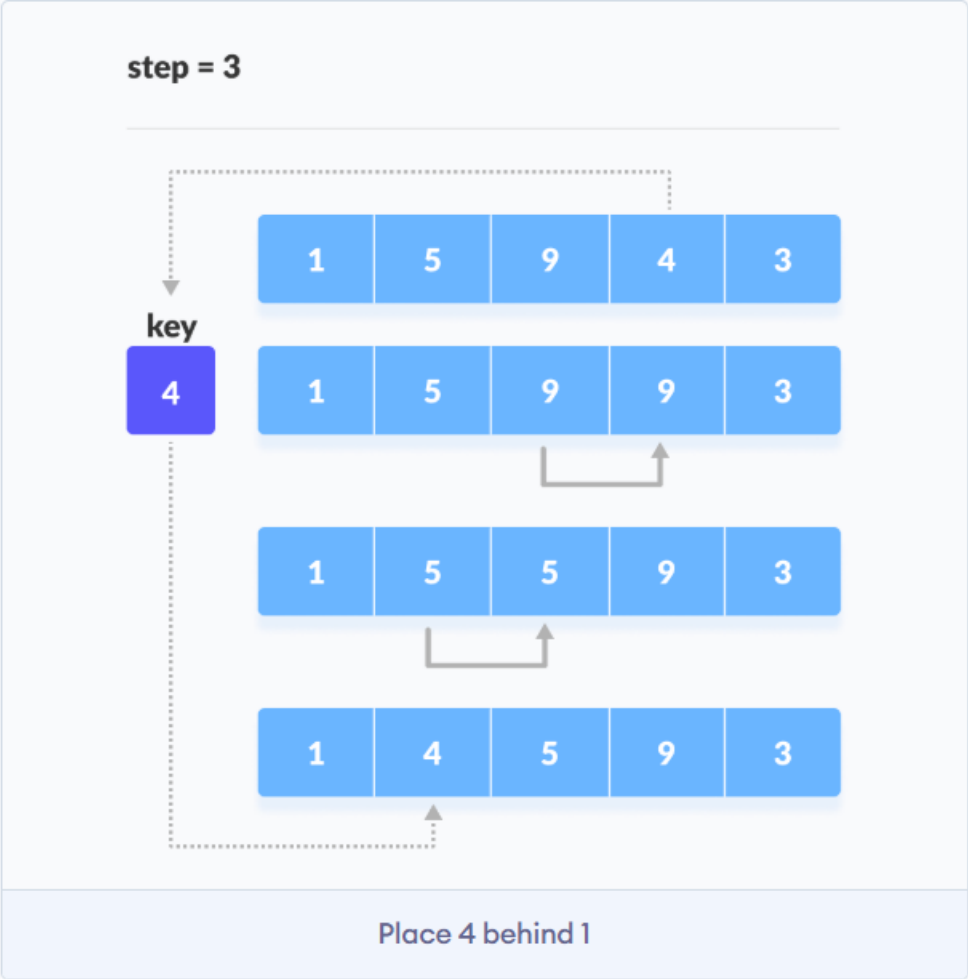
- Now, the first two elements are sorted.
- Take the third element and compare it with the elements on the left of it. Placed it just behind the element smaller than it. If there is no element smaller than it, then place it at the beginning of the array.



Place 1 at the beginning

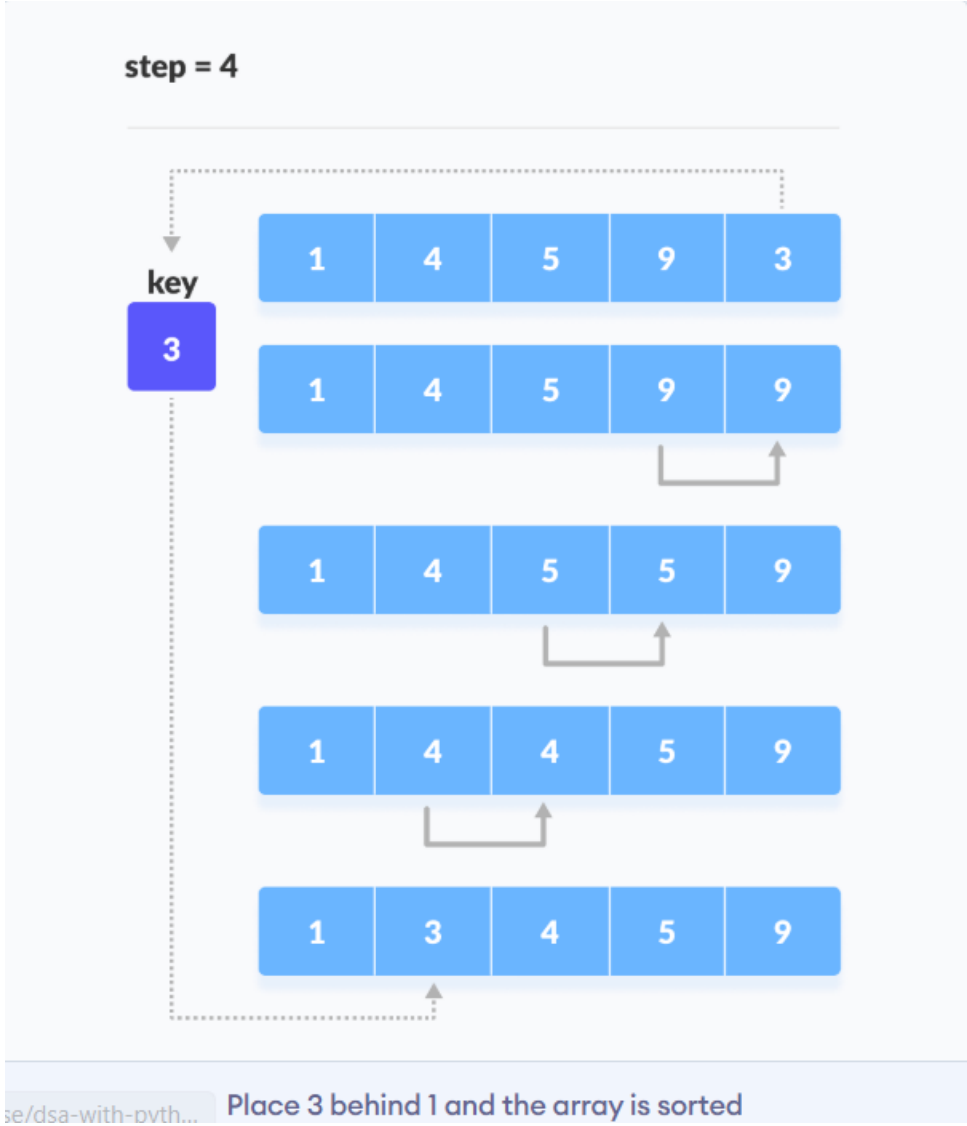
Working of Insertion Sort

- Similarly, place every unsorted element at its correct position.



Working of Insertion Sort

- Similarly, place every unsorted element at its correct position.



Insertion Sort Algorithm

```
insertionSort(array)
  mark first element as sorted
  for each unsorted element X
    'extract' the element X
    for j <- lastSortedIndex down to 0
      if current element j > X
        move sorted element to the right by 1
    break loop and insert X here
end insertionSort
```

Shell Sort

- Shell sort is a generalized version of the insertion sort algorithm. It first sorts elements that are far apart from each other and successively reduces the interval between the elements to be sorted

```
shellSort(array, size)
  for interval i <- size/2n down to 1
    for each interval "i" in array
        sort all the elements at interval "i"
end shellSort
```

Working of Shell Sort

Radix Sort

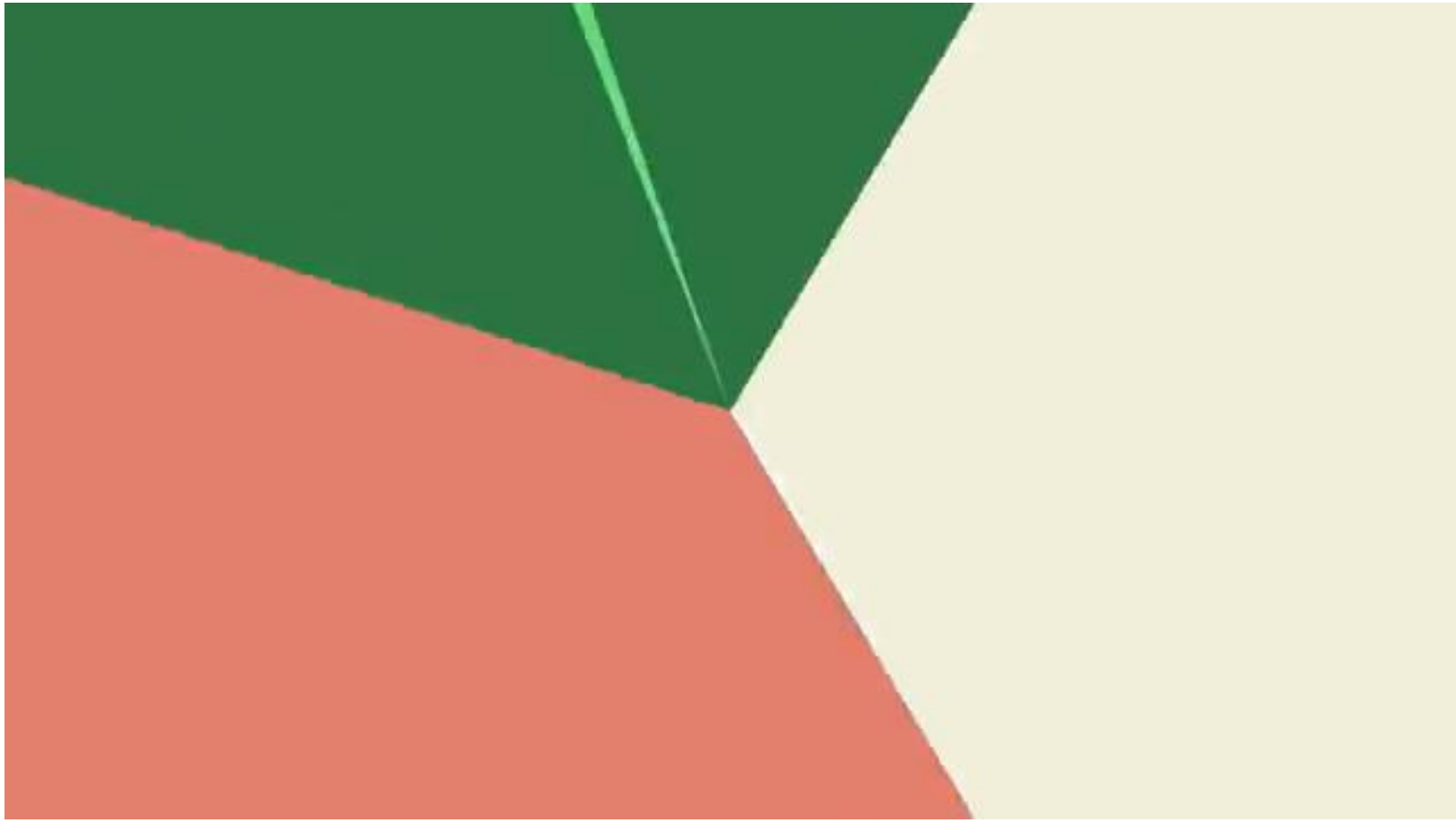
- Radix sort is a sorting algorithm that sorts the elements by first grouping the individual digits of the same place value. Then, sort the elements according to their increasing/decreasing order.
- Suppose, we have an array of 8 elements. First, we will sort elements based on the value of the unit place. Then, we will sort elements based on the value of the tenth place. This process goes on until the last significant place.
- Let the initial array be [121, 432, 564, 23, 1, 45, 788]. It is sorted according to radix sort as shown in the figure below.

Radix Sort Algorithm

```
radixSort(array)
  d <- maximum number of digits in the largest element
  create d buckets of size 0-9
  for i <- 0 to d
    sort the elements according to ith place digits using countingSort

countingSort(array, d)
  max <- find largest element among dth place elements
  initialize count array with all zeros
  for j <- 0 to size
    find the total count of each unique digit in dth place of elements and
    store the count at jth index in count array
  for i <- 1 to max
    find the cumulative sum and store it in count array itself
  for j <- size down to 1
    restore the elements to array
    decrease count of each element restored by 1
```

Working of Radix Sort



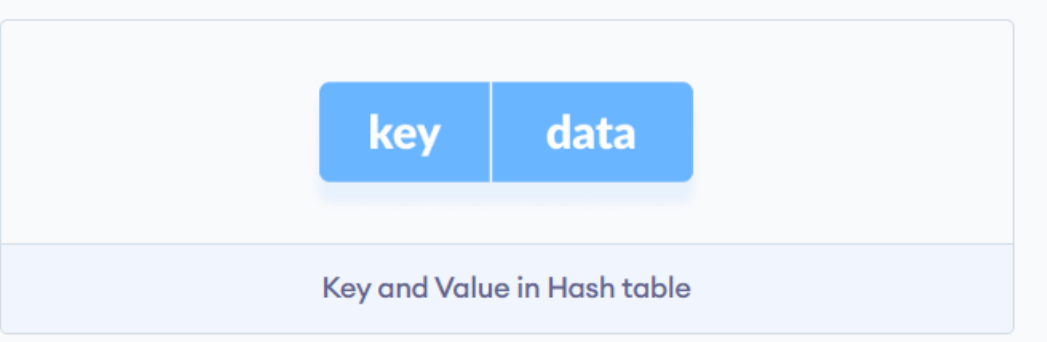
Hashing

- Hashing is the process of converting input data of any size into a fixed-size value, usually for the purpose of fast data retrieval.
- Hash functions are used to map the input data (often called a “key”) to a fixed-size string of bytes.

Hash Table

The Hash table data structure stores elements in key-value pairs where

- **Key**- unique integer that is used for indexing the values
- **Value** - data that are associated with keys.



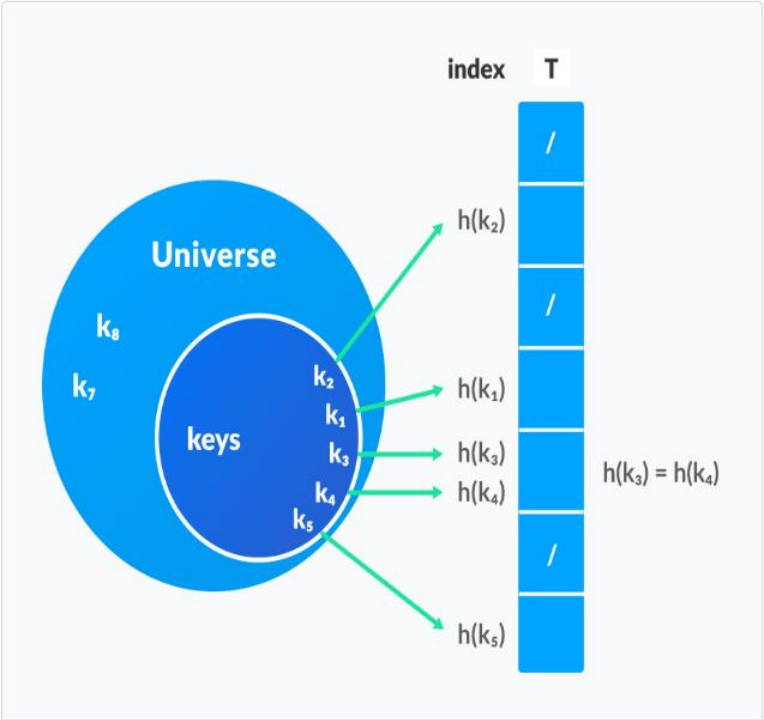
Hashing (Hash Function)

- In a hash table, a new index is processed using the keys.
- And, the element corresponding to that key is stored in the index. This process is called hashing.

Example for hashing

Let k be a key and h(x) be a hash function.

- Here, h(k) will give us a new index to store the element linked with k



Hash table Representation

Hash Collision

When the hash function generates the same index for multiple keys, there will be a conflict (what value to be stored in that index). This is called a hash collision.

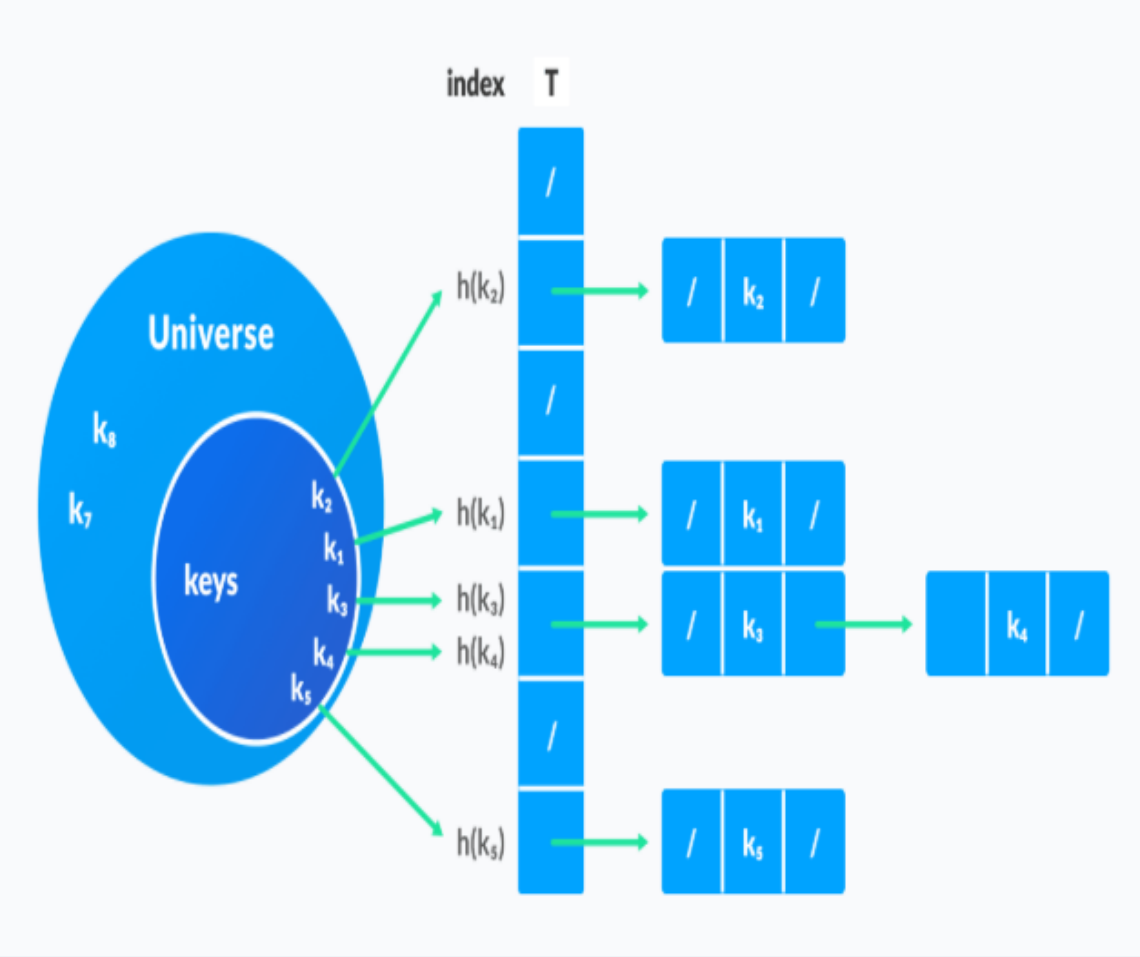
We can resolve the hash collision using one of the following techniques.

- Collision resolution by **separate chaining**
- **Open Addressing:** Linear/Quadratic Probing and Double Hashing

Collision resolution by chaining

- In chaining, if a hash function produces the same index for multiple elements, these elements are stored in the same index by using a doubly-linked list.
- If j is the slot for multiple elements, it contains a pointer to the head of the list of elements. If no element is present, j contains NIL

Collision resolution by chaining



Collision Resolution using chaining

Pseudo code for operations:

```
chainedHashSearch(T, k)
  return T[h(k)]
chainedHashInsert(T, x)
  T[h(x.key)] = x //insert at the head
chainedHashDelete(T, x)
  T[h(x.key)] = NIL
```

Hash Functions

- A good hash function may not prevent the collisions completely however it can reduce the number of collisions

Methods

- Division Method
- Multiplication Method
- Universal Hashing

Division Method

- If k is a key and m is the size of the hash table, the hash function h() is calculated as:

$$h(k) = k \bmod m$$

For example, If the size of a hash table is 10 and k = 112 then h(k) = 112 mod 10 = 2.

- The value of m must not be the powers of 2.
- This is because the powers of 2 in binary format are 10, 100, 1000, …. When we find k mod m, we will always get the lower order p-bits.

Division Method

- if m = 22, k = 17, then h(k) = 17 mod 22 = 10001 mod 100 = 01
- if m = 23, k = 17, then h(k) = 17 mod 22 = 10001 mod 100 = 001
- if m = 24, k = 17, then h(k) = 17 mod 22 = 10001 mod 100 = 0001
- if m = 2p, then h(k) = p lower bits of m

Multiplication Method

- $h(k) = \lfloor m(kA \bmod 1) \rfloor$

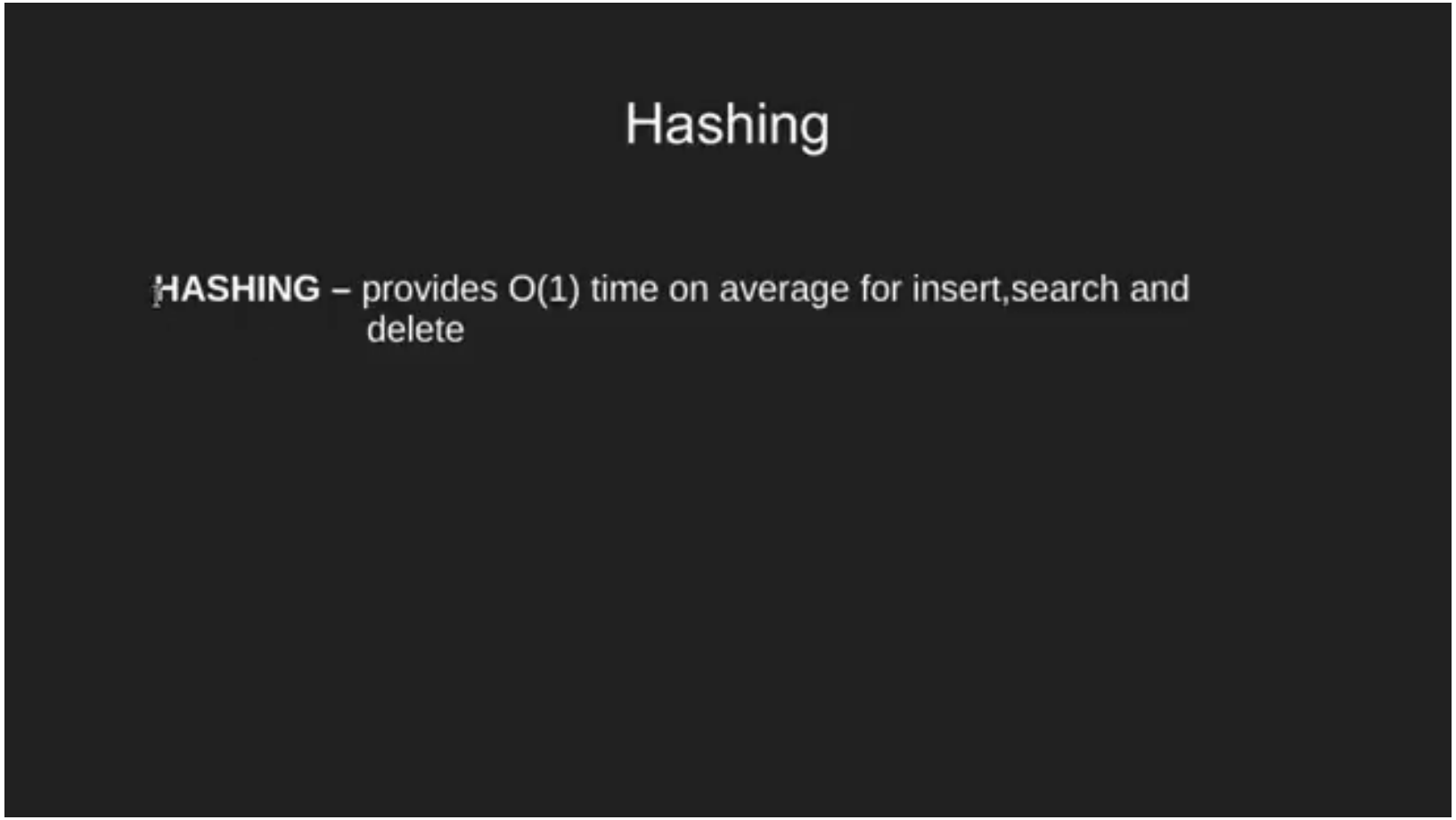
where,

  - $kA \bmod 1$ gives the fractional part kA,
  - $\lfloor\ \rfloor$ gives the floor value
  - A is any constant. The value of A lies between 0 and 1. But, an optimal choice will be $\approx (\sqrt{5}-1)/2$ suggested by Knuth.

Universal Hashing

- In Universal hashing, the hash function is chosen at random independent of keys.

Working of Collision resolution by chaining



Open Addressing

- Unlike chaining, open addressing doesn't store multiple elements into the same slot. Here, each slot is either filled with a single key or left NIL.
- Different techniques used in open addressing are
  - Linear Probing
  - Quadratic Probing
  - Double hashing

Linear Probing

In linear probing, collision is resolved by checking the next slot.

$$h(k, i) = (h'(k) + i) \bmod m$$

where

$i = \{0, 1, ...\}$

$h'(k)$ is a new hash function

I

f a collision occurs at $h(k, 0)$, then $h(k, 1)$ is checked. In this way, the value of i is incremented linearly.

The problem with linear probing is that a cluster of adjacent slots is filled. When inserting a new element, the entire cluster must be traversed. This adds to the time required to perform operations on the hash table.

Quadratic Probing

It works similar to linear probing but the spacing between the slots is increased (greater than one) by using the following relation.

$$h(k, i) = (h'(k) + c1i + c2i2) \bmod m$$

where,

c1 and c2 are positive auxiliary constants,

i = {0, 1, ….}

Double hashing

If a collision occurs after applying a hash function h(k), then another hash function is calculated for finding the next slot.

$$h(k, i) = (h1(k) + ih2(k)) \bmod m$$

Rehashing

- Rehashing is the process of resizing a hash table and redistributing its elements when the current size of the table no longer efficiently accommodates the number of elements it contains.
- The primary goal of rehashing is to maintain a low load factor, which is the ratio of the number of stored elements to the total number of buckets in the hash table.
- A low load factor helps ensure that the hash table remains efficient in terms of time complexity for insertion, retrieval, and deletion operations

Rehashing

- **Check the Load Factor:** Periodically or after each insertion operation, the hash table checks its load factor. If the load factor exceeds a predefined threshold (often around 0.7 or 0.8), it indicates that the table is becoming crowded, and rehashing is needed.
- **Create a New Hash Table**: A new, larger hash table (usually with double the number of buckets) is created. The number of buckets is increased to reduce the load factor and make the table more efficient.
- **Rehashing Process**: Each element in the old hash table is rehashed, meaning their keys are mapped to new bucket positions in the larger table using the updated hash function. This process redistributes the key-value pairs among the new buckets.
- **Transfer Elements:** The key-value pairs are transferred from the old table to the new table based on their new hash values. This involves copying the data from the old table to the appropriate locations in the new table.
- **Update References:** Any references or pointers to the old hash table are updated to point to the new hash table.
- **Dispose of the Old Table:** Once all elements have been transferred, the old hash table can be deallocated or discarded

Extendible hashing

- Extendible hashing is a dynamic hashing technique used in computer science and database systems to efficiently organize and search data.
- It is designed to provide a compromise between static hashing (which requires a fixed number of buckets) and dynamic hashing (which may involve frequent rehashing).
- Extendible hashing dynamically adapts the number of buckets as data grows, minimizing the overhead associated with rehashing.

Extendible hashing

- **Initialization:** Initially, there is a single directory that contains a fixed number of empty buckets. The directory acts as an index or directory of the available buckets.
- **Hashing:** When a new key-value pair needs to be inserted or searched for, a hash function is applied to the key to determine the bucket in which the pair should be stored or searched. The hash function generates an index into the directory.
- **Directory Structure:** The directory contains entries, each of which corresponds to a specific bucket. Initially, all entries point to the same bucket. These pointers are usually represented as binary strings or integers.
- **Bucket Splitting:** When a bucket becomes full (i.e., it reaches a predefined load factor), it is split into two new buckets. This splitting operation is done by adding a single bit to the binary string associated with the directory entry that pointed to the full bucket. This effectively doubles the number of buckets available for storing data

Extendible hashing

- **Directory Expansion:** As buckets are split and new ones are created, the directory's structure grows dynamically. The directory entries are updated to reflect the new bucket locations as they are created.
- **Search and Insertion:** When searching for a key or inserting a new key-value pair, the hash function generates an index into the directory. The directory entry at that index is used to determine the appropriate bucket to search or insert into.
- **Merging:** Over time, if some buckets become empty due to deletions or if the directory becomes sparsely populated, extendible hashing allows for merging. This involves reducing the number of buckets by removing a bit from directory entries and merging adjacent buckets to maintain a balanced structure.

Extendible hashing Advantages

- It adapts dynamically to the data distribution, avoiding the need for frequent rehashing.
- It provides a good balance between space usage and search efficiency.
- It offers predictable performance for search, insertion, and deletion operations.